Report

Elise CHABRERIE

Summary **2**
Sequential Program **3**
Principle 3
C implementation 3
C++ implementation 6
Tests 9
Complexity 9
Performances 10
Concurrent Program **14**
1) Brute-force algorithm 14
Principle 14
Implementation with OpenMP 14
Complexity 17
Performances 18
2) Sliding window algorithm with file reading by multiple threads 26
Principle 26
Implementation with OpenMP 26
Performances 27
3) Sliding window algorithm with parallel reading and calculation of the sequence 29
Principle 29
Performances 29
Conclusion **30**

Summary

Here is the report on the first assignment :)

Sequential Program

Principle

There are a number of houses, and each house gives a certain number of candies (candy[i]). The child must visit a continuous sequence of houses without exceeding a maximum number of candies (maxCandy). The goal is to find the sequence of consecutive houses that gives the maximum number of candies without exceeding the limit (if several sequences are possible, it is the sequence that starts with the house with the lowest number that is returned, and if no sequence is possible, "Don't go here" must be displayed).

The algorithm's principle is to use a sliding window. We use a window of consecutive houses where the sum of the candies (distributed by the household) is less than or equal to maxCandy. We slide this window along the array to test all possible sequences. We expand this window towards houses with the highest index as much as possible, and when this is no longer possible because the sum exceeds the limit, we shrink the window by removing houses with lower indices.

C implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>




/*
Reads a file and fills the candy array.
Parameters:
  - filename : name of the input file
  - homes    : pointer to the number of houses
  - maxCandy : pointer to the maximum allowed candy
  - candy    : pointer to the candy array
Returns:
  0  if successful
 -1  if an error occurred
*/
int read_input(const char *filename, int *homes, int *maxCandy, int
**candy) {

    FILE *f = fopen(filename, "r");
```

```
        if (!f) {
            printf("Error : cannot open %s\n", filename);
            return -1;
        }

        fscanf(f, "%d", homes);
        fscanf(f, "%d", maxCandy);

        if (*homes <= 0) {
            fclose(f);
            return 0;
        }

        *candy = (int*) malloc((*homes) * sizeof(int));
        if (!*candy) {
            printf("Error : memory allocation failed\n");
            fclose(f);
            return -1;
        }

        for (int i = 0; i < *homes; i++)
            fscanf(f, "%d", &((*candy)[i]));

        fclose(f);
        return 0;
    }



    /*
    Finds the best sequence of consecutive houses to visit to collect
the maximum candy without exceeding maxCandy.
    Parameters:
      - candy     : array of candies per house
      - homes     : number of houses
      - maxCandy  : maximum allowed candy
      - bestStart : pointer to the index of the first house
      - bestEnd   : pointer to the index of the last house
      - bestSum   : pointer to the maximum sum found
    Returns:
```

```
        1 if a sequence is found
        0 otherwise
      */
      int best_sequence(int *candy, int homes, int maxCandy, int
*bestStart, int *bestEnd, int *bestSum) {

         *bestSum = -1;
         *bestStart = -1;
         *bestEnd = -1;

         if (homes <= 0) return 0;

         int sum = 0;
         int start = 0;

         for (int end = 0; end < homes; end++) {
             sum += candy[end];

             // Shrink the window (on the side of the houses with the
lowest index) if sum exceeds maxCandy
             while (sum > maxCandy && start <= end) {
                 sum -= candy[start];
                 start++;
             }

             // Update if the sum is strictly greater than bestSum and is
correct
             if (sum <= maxCandy && sum > *bestSum && start <= end) {

                 *bestSum = sum;
                 *bestStart = start;
                 *bestEnd = end;

                 // If we reach exactly maxCandy, we can stop, we won't
find a better solution starting at a lower house index
                 if (*bestSum == maxCandy)
                     return 1;
             }
         }
```

```
        return (*bestSum != -1) ? 1 : 0;
    }




    /*
    Prints the final result.
    Parameters:
      - bestStart : index of the first house
      - bestEnd   : index of the last house
      - bestSum   : total candy collected
    */
    void print_result(int bestStart, int bestEnd, int bestSum) {
        if (bestSum == -1) {
            printf("Don't go here\n");
        } else {
            // The houses indexes that from 1, so we add 1 to convert the
indexes
            printf("Start at home %d and go to home %d getting %d pieces
of candy\n",
                    bestStart + 1, bestEnd + 1, bestSum);
        }
    }
```

Fig.1 The trick_or_treat.c code

C++ implementation

```
    #include <iostream>
    #include <fstream>
    #include <vector>

    /*
    Reads a file and fills the candy vector.
    Parameters:
      - filename : reference to the name of the input file
      - homes    : reference to the number of houses
      - maxCandy : reference to the maximum allowed candy
      - candy    : reference to the vector of candies
    Returns:
```

```
        true if successful
        false if an error occurred
      */
      bool read_input(const std::string &filename, int &homes, int
&maxCandy, std::vector<int> &candy) {

         std::ifstream file(filename);
         if (!file) {
             std::cerr << "Error: could not open " << filename <<
std::endl;
             return false;
         }

         file >> homes;
         file >> maxCandy;
         if (homes <= 0) {
             file.close();
             return true;
         }

         candy.resize(homes);
         for (int i = 0; i < homes; i++) {
             file >> candy[i];
         }

         file.close();
         return true;
      }

      /*
      Finds the best sequence of consecutive houses to collect the maximum
candy without exceeding maxCandy.
      Parameters:
        - candy     : vector of candies per house
        - homes     : number of houses
        - maxCandy  : maximum allowed candy
        - bestStart : reference to the index of the first house
        - bestEnd   : reference to the index of the last house
        - bestSum   : reference to the maximum sum found
      Returns:
```

```
        true if a sequence is found
        false otherwise
      */
      bool best_sequence(const std::vector<int> &candy, int homes, int
maxCandy, int &bestStart, int &bestEnd, int &bestSum) {

         bestSum = -1;
         bestStart = -1;
         bestEnd = -1;

         if (homes <= 0) return false;

         int sum = 0;
         int start = 0;

         for (int end = 0; end < homes; ++end) {
             sum += candy[end];

             while (sum > maxCandy && start <= end) {
                 sum -= candy[start];
                 start++;
             }

             if (sum <= maxCandy && sum > bestSum && start <= end) {
                 bestSum = sum;
                 bestStart = start;
                 bestEnd = end;

                 // optimal, so we stop here
                 if (bestSum == maxCandy)
                     return true;
             }
         }

         return (bestSum != -1);
      }

      /*
      Prints the final result.
      Parameters:
```

```
        - bestStart : index of the first house
        - bestEnd   : index of the last house
        - bestSum   : total candy collected
      */
      void print_result(int bestStart, int bestEnd, int bestSum) {
         if (bestSum == -1)
             std::cout << "Don't go here\n";
         else {
             std::cout << "Start at home " << (bestStart + 1) << " and go
to home " << (bestEnd + 1) << " getting " << bestSum << " pieces of
candy\n";
         }
      }
```

Fig.2 The trick_or_treat.cpp code

We have broken the problem down into three sub-functions which respectively read the data from the txt file, find the optimal sequence of houses, and print the optimal result.

We note that we have implemented early stopping which ensures that the first sequence allowing us to get maxCandy candies is returned and the program does not traverse the entire vector homes.

Tests

We created an inout_generator.cpp file that generates input_x.txt files for x different numbers of houses up to 10,000 (and by default, maxCandy is 500). We also created input_x_pirecas.txt, for x different numbers of houses, which are cases where early stopping is not possible. This allowed us to test our program on it.

We also verified, using small manual examples, that specific cases worked: when there are multiple solutions and the sum needs to be returned starting with the smallest index, when all the houses give more candy than maxCandy, when maxCandy is 0 (if we are looking for 0 candy then we still look, even though it's not necessary we want someone to give 0), etc.

Complexity

Time complexity

The algorithm is based on a sliding window to find the consecutive sequence of houses that yields the most candies without exceeding maxCandy. The outer loop iterates through each house (end) once, and the inner loop (while) advances the start of the window (start) when the

sum exceeds maxCandy. Thus, $T_{trickortreat}(n) = T_{readinput}(n) + T_{bestsequence}(n) + T_{printresult}(n) = O(n) + \sum_{end=0}^{n-1} (O(1) + O(while)) + O(1)$. However, each house can only be removed at most once by advancing the start of the window, so $\sum_{end=0}^{n-1} O(while) \leq n$. Finally, $O(trickortreat) = O(3n) = O(n)$, the time complexity is linear.

Space complexity

The spatial complexity of the algorithm is determined by the data structures used. The algorithm primarily stores the candy vector containing the number of candies for each house, which requires a storage space proportional to the number of houses, i.e., n = homes. In addition, there are a few integer variables (sum, start, bestStart, bestEnd, bestSum), whose space is constant and independent of n. No additional arrays or recursive structures are used. Thus, the total space complexity is linear.

Performances

Time performances

We developed a benchmark script to measure the execution times of our algorithm. For each input size (from 100 to 10,000 houses), we perform 20 runs with different random files. We display the mean, median, standard deviation, and coefficient of variation (CV) of the execution times. The benchmark tests each component (read, compute, display) separately, as well as the entire pipeline. The "Time" value represents the actual elapsed time, while "CPU" indicates the actual CPU time used.

```
computer@computer:~/Bureau/Elise_asuppr/Nvidia/trick_or_treat/in_cpp$ ./benchmark
Benchmark                                    Time             CPU   Iterations
-----------------------------------------------------------------------------
BM_ReadInput/100/repeats:20_mean          12715 ns        12683 ns           20
BM_ReadInput/100/repeats:20_median        12418 ns        12416 ns           20
BM_ReadInput/100/repeats:20_stddev         1079 ns          968 ns           20
BM_ReadInput/100/repeats:20_cv             8.49 %          7.63 %           20
BM_ReadInput/500/repeats:20_mean          54421 ns        53989 ns           20
BM_ReadInput/500/repeats:20_median        53110 ns        53099 ns           20
BM_ReadInput/500/repeats:20_stddev         8827 ns         8368 ns           20
BM_ReadInput/500/repeats:20_cv            16.22 %         15.50 %           20
BM_ReadInput/1000/repeats:20_mean         85267 ns        84699 ns           20
BM_ReadInput/1000/repeats:20_median       84617 ns        84610 ns           20
BM_ReadInput/1000/repeats:20_stddev        8946 ns         7056 ns           20
BM_ReadInput/1000/repeats:20_cv           10.49 %          8.33 %           20
BM_ReadInput/5000/repeats:20_mean        390579 ns       390531 ns           20
BM_ReadInput/5000/repeats:20_median      388817 ns       388783 ns           20
BM_ReadInput/5000/repeats:20_stddev       24057 ns        24070 ns           20
BM_ReadInput/5000/repeats:20_cv            6.16 %          6.16 %           20
BM_ReadInput/10000/repeats:20_mean       776242 ns       775923 ns           20
BM_ReadInput/10000/repeats:20_median     783037 ns       782747 ns           20
BM_ReadInput/10000/repeats:20_stddev      45348 ns        45307 ns           20
BM_ReadInput/10000/repeats:20_cv           5.84 %          5.84 %           20
```

Fig.3 : Benchmark of read_input for inputs of different sizes

BM_BestSequence/100/repeats:20_mean	954 ns	954 ns
BM_BestSequence/100/repeats:20_median	1013 ns	1013 ns
BM_BestSequence/100/repeats:20_stddev	276 ns	275 ns
BM_BestSequence/100/repeats:20_cv	28.87 %	28.87 %
BM_BestSequence/500/repeats:20_mean	3518 ns	3517 ns
BM_BestSequence/500/repeats:20_median	4445 ns	4445 ns
BM_BestSequence/500/repeats:20_stddev	2238 ns	2238 ns
BM_BestSequence/500/repeats:20_cv	63.62 %	63.62 %
BM_BestSequence/1000/repeats:20_mean	4631 ns	4630 ns
BM_BestSequence/1000/repeats:20_median	2586 ns	2585 ns
BM_BestSequence/1000/repeats:20_stddev	4972 ns	4971 ns
BM_BestSequence/1000/repeats:20_cv	107.37 %	107.36 %
BM_BestSequence/5000/repeats:20_mean	3864 ns	3864 ns
BM_BestSequence/5000/repeats:20_median	3411 ns	3410 ns
BM_BestSequence/5000/repeats:20_stddev	2910 ns	2910 ns
BM_BestSequence/5000/repeats:20_cv	75.30 %	75.30 %
BM_BestSequence/10000/repeats:20_mean	3615 ns	3608 ns
BM_BestSequence/10000/repeats:20_median	2094 ns	2094 ns
BM_BestSequence/10000/repeats:20_stddev	4283 ns	4282 ns
BM_BestSequence/10000/repeats:20_cv	118.47 %	118.70 %

Fig.4 : Benchmark of best_sequence for inputs of different sizes

BM_PrintResult/100/repeats:20_mean	299 ns	299 ns
BM_PrintResult/100/repeats:20_median	300 ns	299 ns
BM_PrintResult/100/repeats:20_stddev	30.5 ns	30.5 ns
BM_PrintResult/100/repeats:20_cv	10.20 %	10.20 %
BM_PrintResult/500/repeats:20_mean	331 ns	325 ns
BM_PrintResult/500/repeats:20_median	286 ns	286 ns
BM_PrintResult/500/repeats:20_stddev	109 ns	93.4 ns
BM_PrintResult/500/repeats:20_cv	33.01 %	28.75 %
BM_PrintResult/1000/repeats:20_mean	282 ns	282 ns
BM_PrintResult/1000/repeats:20_median	276 ns	276 ns
BM_PrintResult/1000/repeats:20_stddev	42.5 ns	42.4 ns
BM_PrintResult/1000/repeats:20_cv	15.06 %	15.05 %
BM_PrintResult/5000/repeats:20_mean	329 ns	325 ns
BM_PrintResult/5000/repeats:20_median	302 ns	301 ns
BM_PrintResult/5000/repeats:20_stddev	62.9 ns	58.4 ns
BM_PrintResult/5000/repeats:20_cv	19.11 %	17.98 %
BM_PrintResult/10000/repeats:20_mean	334 ns	320 ns
BM_PrintResult/10000/repeats:20_median	297 ns	297 ns
BM_PrintResult/10000/repeats:20_stddev	93.3 ns	70.3 ns
BM_PrintResult/10000/repeats:20_cv	27.90 %	21.96 %

Fig.5 : Benchmark of print result for inputs of different sizes

```
BM_FullPipeline/100/repeats:20_mean          38548 ns        33764 ns
BM_FullPipeline/100/repeats:20_median        35243 ns        32885 ns
BM_FullPipeline/100/repeats:20_stddev        11134 ns         6018 ns
BM_FullPipeline/100/repeats:20_cv            28.88 %         17.82 %
BM_FullPipeline/1000/repeats:20_mean        227386 ns       203352 ns
BM_FullPipeline/1000/repeats:20_median      201881 ns       194099 ns
BM_FullPipeline/1000/repeats:20_stddev       56941 ns        33848 ns
BM_FullPipeline/1000/repeats:20_cv           25.04 %         16.64 %
BM_FullPipeline/5000/repeats:20_mean       1050978 ns       997466 ns
BM_FullPipeline/5000/repeats:20_median      998516 ns       974891 ns
BM_FullPipeline/5000/repeats:20_stddev      161106 ns       109028 ns
BM_FullPipeline/5000/repeats:20_cv           15.33 %         10.93 %
BM_FullPipeline/10000/repeats:20_mean      1991217 ns      1940822 ns
BM_FullPipeline/10000/repeats:20_median    2029131 ns      1975228 ns
BM_FullPipeline/10000/repeats:20_stddev     195816 ns       180886 ns
BM_FullPipeline/10000/repeats:20_cv           9.83 %          9.32 %
```

Fig.6 : Benchmark of the full pipeline for inputs of different sizes

Figure 3 shows that `read_input` has a slightly sublinear complexity (approximately 12,418 ns for 100 houses and approximately 783,037 ns for 10,000 houses, resulting in a ratio of ×63 instead of the expected ×100 if it were perfectly linear). This function represents the dominant portion of the pipeline but remains within acceptable limits.

The median time for best_sequence increases with input size, but not in a perfectly linear fashion as seen in Fig.4. The very high coefficient of variation (cv ranges from 29% to 118% depending on the input size) is explained by the early stopping in the sliding window algorithm : depending on the random data, the solution may be found immediately or require a complete traversal. Despite this variance, the average time remains low.

Finally, the complete pipeline confirms that the workload is dominated by reading the file, with display and even the calculation phases being negligible in comparison.

```
computer@computer:~/Bureau/Elise_asuppr/Nvidia/trick_or_treat/in_cpp$ perf stat ./test
Starting test...

Start at home 1 and go to home 10000 getting 1 pieces of candy

=== RESULTS ===
File: ../input_pirecas.txt
Homes: 10000
Max candy: 1000
Result: Start=1, End=10000, Sum=1
Total time: 0 ms

 Performance counter stats for './test':

              3,06 msec task-clock                       #    0,850 CPUs utilized
                 0      context-switches                 #    0,000 /sec
                 0      cpu-migrations                   #    0,000 /sec
               141      page-faults                      #   46,076 K/sec
        10 184 138      cycles                           #    3,328 GHz
        15 683 713      instructions                     #    1,54  insn per cycle
         2 979 000      branches                         #  973,468 M/sec
            24 090      branch-misses                    #    0,81% of all branches

       0,003602247 seconds time elapsed

       0,001844000 seconds user
       0,001844000 seconds sys
```

Fig.7 : Performance statistics

We tested the algorithm on an input called "input_10000_pirecas.txt", which requires traversing 10,000 houses, representing one of the worst-case scenarios for the algorithm. We observe that 0.85 CPUs are used, which is acceptable for a single-threaded program. Furthermore, the instructions per cycle is 1.54, indicating good CPU utilization. Additionally, the branch-miss rate is low. There doesn't appear to be a bottleneck.

Space performances

```
computer@computer:~/Bureau/Elise_asuppr/Nvidia/trick_or_treat/in_cpp$ perf stat -e L1-dcache-loads,L1-dcache-load-misses,LLC-loads,LLC-load-misses ./test
Testing with file: ../input_pirecas.txt
Starting test...

Start at home 1 and go to home 10000 getting 1 pieces of candy

=== RESULTS ===
File: ../input_pirecas.txt
Homes: 10000
Max candy: 1000
Result: Start=1, End=10000, Sum=1
Total time: 1 ms

 Performance counter stats for './test':

         4 572 245      L1-dcache-loads
            72 576      L1-dcache-load-misses           #    1,59% of all L1-dcache accesses
            22 414      LLC-loads
             6 659      LLC-load-misses                 #   29,71% of all LL-cache accesses

       0,004299943 seconds time elapsed

       0,002587000 seconds user
       0,001724000 seconds sys
```

Fig.8 : Cache performance

The Fig.8 shows a low cache-miss rate (1.59%) for accesses to the nearest L1 cache, and these are the most frequent accesses. On the other hand, there is a quite high proportion of misses for accesses to the last cache, but it is accessed much less frequently. Accesses (to the input file) and writes (to Candy, in particular) are contiguous, this spatial locality allows for efficient use of cache lines, and this contiguity helps with prefetching.

Concurrent Program

1) Brute-force algorithm

Principle

The sliding window algorithm is fundamentally sequential. At each step, the current sum depends on the previous sum and the window's position (start and end). To advance to end, the accumulated sum must be known, and start must be adjusted if the sum exceeds the maxCandy limit. This dependency between steps makes parallel execution of the loop iterations impossible, as each iteration cannot be calculated independently of the others. Thus, even with multiple cores, this loop cannot be divided into parallel sub-sections without completely changing the algorithm's logic.

Therefore, we decided to test a brute-force algorithm and optimize it for parallel execution. The algorithm consists of testing all possible sub-sequences among the houses and calculating their candy sum. Each sub-sequence starts at a house with index i and extends to a house j ≥ i, stopping if the sum exceeds the maxCandy limit. Because each subsequence is independent, the outer loop can be easily parallelized : each thread processes a set of different starting points (homes[i]) and locally computes the best possible sequence that begins at that starting point. Then, a critical section or reduction combines the local results to determine the best overall sequence. This approach allows for efficient use of multiple cores, even though the algorithm is more time-consuming than the sliding window.

Implementation with OpenMP

```
      bool best_sequence_parallel(const std::vector<int> &candy, int
homes, int maxCandy,
                                int &bestStart, int &bestEnd, int
&bestSum) {

        bestSum = -1;
        bestStart = -1;
        bestEnd = -1;

        // OpenMP parallel region
        #pragma omp parallel
        {
            int localBestSum = -1;
            int localStart = -1;
```

```
            int localEnd = -1;

            // Distribute starting indices across threads
            #pragma omp for
            for (int i = 0; i < homes; i++) {

                int sum = 0;

                for (int j = i; j < homes; j++) {

                    sum += candy[j];

                    // Early exit if the sum exceeds maxCandy
                    if (sum > maxCandy)
                        break;

                    if (sum > localBestSum) {
                        localBestSum = sum;
                        localStart = i;
                        localEnd = j;
                    }

                    // Early exit if we reach maxCandy exactly
                    // (Adding further homes would iply that the next
homes give zero candy but it does not improve the result)
                    if (sum == maxCandy)
                        break;
                }
            }

            // Manual reduction to update the global best result
            // If there is a tie on sum, choose the sequence with the
smaller starting index
            #pragma omp critical
            {
                if (localBestSum > bestSum ||
                    (localBestSum == bestSum && localStart < bestStart))
{

                    bestSum = localBestSum;
```

```
                    bestStart = localStart;
                    bestEnd = localEnd;
                }
            }
        }

        return (bestSum != -1);
    }
```

Fig.9 : The brute search program best_sequence_parallel with open mp optimizations.

The first thread that finds maxCandy does not necessarily have the lowest possible starting index.

BM_BestSequenceParallel/1/100/repeats:20_mean	5026 ns	4950 ns	20
BM_BestSequenceParallel/1/100/repeats:20_median	4935 ns	4922 ns	20
BM_BestSequenceParallel/1/100/repeats:20_stddev	775 ns	554 ns	20
BM_BestSequenceParallel/1/100/repeats:20_cv	15.43 %	11.19 %	20
BM_BestSequenceParallel/2/100/repeats:20_mean	6576 ns	6489 ns	20
BM_BestSequenceParallel/2/100/repeats:20_median	6279 ns	6277 ns	20
BM_BestSequenceParallel/2/100/repeats:20_stddev	1457 ns	1284 ns	20
BM_BestSequenceParallel/2/100/repeats:20_cv	22.15 %	19.79 %	20
BM_BestSequenceParallel/4/100/repeats:20_mean	11007 ns	9840 ns	20
BM_BestSequenceParallel/4/100/repeats:20_median	10005 ns	8414 ns	20
BM_BestSequenceParallel/4/100/repeats:20_stddev	4482 ns	3430 ns	20
BM_BestSequenceParallel/4/100/repeats:20_cv	40.72 %	34.86 %	20
BM_BestSequenceParallel/8/100/repeats:20_mean	130004 ns	38289 ns	20
BM_BestSequenceParallel/8/100/repeats:20_median	129994 ns	37559 ns	20
BM_BestSequenceParallel/8/100/repeats:20_stddev	21026 ns	1590 ns	20
BM_BestSequenceParallel/8/100/repeats:20_cv	16.17 %	4.15 %	20
BM_BestSequenceParallel/16/100/repeats:20_mean	277582 ns	41385 ns	20
BM_BestSequenceParallel/16/100/repeats:20_median	233596 ns	41521 ns	20
BM_BestSequenceParallel/16/100/repeats:20_stddev	88545 ns	3459 ns	20
BM_BestSequenceParallel/16/100/repeats:20_cv	31.90 %	8.36 %	20

Fig.10 : Benchmark of best parallel sequence with a dynamic schedule for different numbers of threads for an input of size 100

BM_BestSequenceParallel/1/10000/repeats:20_mean	442870 ns	442795 ns	20
BM_BestSequenceParallel/1/10000/repeats:20_median	431217 ns	431187 ns	20
BM_BestSequenceParallel/1/10000/repeats:20_stddev	42246 ns	42216 ns	20
BM_BestSequenceParallel/1/10000/repeats:20_cv	9.54 %	9.53 %	20
BM_BestSequenceParallel/2/10000/repeats:20_mean	470664 ns	458638 ns	20
BM_BestSequenceParallel/2/10000/repeats:20_median	450520 ns	450503 ns	20
BM_BestSequenceParallel/2/10000/repeats:20_stddev	74953 ns	47663 ns	20
BM_BestSequenceParallel/2/10000/repeats:20_cv	15.93 %	10.39 %	20
BM_BestSequenceParallel/4/10000/repeats:20_mean	413300 ns	350223 ns	20
BM_BestSequenceParallel/4/10000/repeats:20_median	271981 ns	269153 ns	20
BM_BestSequenceParallel/4/10000/repeats:20_stddev	391342 ns	204318 ns	20
BM_BestSequenceParallel/4/10000/repeats:20_cv	94.69 %	58.34 %	20
BM_BestSequenceParallel/8/10000/repeats:20_mean	402841 ns	184864 ns	20
BM_BestSequenceParallel/8/10000/repeats:20_median	414253 ns	189777 ns	20
BM_BestSequenceParallel/8/10000/repeats:20_stddev	53509 ns	16687 ns	20
BM_BestSequenceParallel/8/10000/repeats:20_cv	13.28 %	9.03 %	20
BM_BestSequenceParallel/16/10000/repeats:20_mean	614016 ns	111440 ns	20
BM_BestSequenceParallel/16/10000/repeats:20_median	585700 ns	111466 ns	20
BM_BestSequenceParallel/16/10000/repeats:20_stddev	132351 ns	9588 ns	20
BM_BestSequenceParallel/16/10000/repeats:20_cv	21.56 %	8.60 %	20

Fig. 11 : Benchmark of best parallel sequence with a dynamic schedule for different numbers of threads for an input of size 10000

We tested the native schedule, which provides a portion of consecutive indices, and the dynamic schedule (using `#pragma omp for schedule(dynamic)`). The static schedule gives each thread a fixed block of iterations, but the cost of an iteration `i` depends on the internal loop from `i` to `n-1`. This internal loop is therefore larger for small indices `i`, resulting in a rather unbalanced workload. The dynamic schedule allows for progressive workload distribution. It assigns a few iterations to a thread, and when it finishes, assigns others. In our case, this might seem useful, but it introduces an overhead.

We saw that overall (Fig.10 & Fig.11 compared to Fig.13 and Fig.15), the schedule without `schedule(dynamic)` is better. The static schedule is significantly faster, especially for smaller inputs. Furthermore, the performance is often lower than that of a single thread, even when a relatively small number of threads is used. Dynamic scheduling doesn't seem necessary at all, it seems to add overhead.

Complexity

Time complexity

The parallel algorithm uses two nested loops: the outer loop iterates through each house as the start of sequence i, and the inner loop iterates through subsequent houses to calculate the sum of consecutive candies until it exceeds maxCandy. Thus, in the worst case —where the inner loop iterates through all remaining indices—, $T_{trickortreat}(n) = O(1) + \sum_{i=0}^{n-1} (O(1) + \sum_{j=i}^{n-1} O(1))$ $= \sum_{i=0}^{n-1} O(n-i) = O(\frac{n(n+1)}{2}) = O(n^2)$, so the theoretical sequential time complexity is quadratic.

However, with OpenMP, the outer loop (i) is distributed across p threads. Each thread receives a subset of indices i. Each thread finds the maximum local sum for all sequences starting with one of the indices it is processing. After completing all its iterations i, the thread enters the critical section to compare its local result with the best global result and update the global result if necessary.

The ideal theoretical parallel processing time, if we have p threads, is $T_p(n) \approx \frac{Tseq(n)}{p} = \frac{O(n^2)}{p}$. We neglect overheads here, such as thread creation, synchronization, memory access, allocation by openmp, etc., which are of a considerable order of magnitude. Each thread receives approximately n/p indices i from the outer loop and therefore has a processing complexity of approximately $\frac{O(n^2)}{p}$.

Amdahl's law states that $Speedup(p) = \frac{1}{S + \frac{P}{P}}$. Theoretically, the total processing time is Tp(n) = an^2+ bn + c where b corresponds to reading, an^2 to the double loop of best_sequence, and c to constant operations (for the displaying for exemple). The sequential part of the program consists of Ts(n) = T(read_input) + T(print_result) + T(critic_section_openmp) = bn + c and the parallel part consists of Tp(n) = an^2. Thus, $S(n) = sequential\ time\ /\ total\ time \approx \frac{bn+c}{an^2+bn+c}$ and P(n)≈ $\frac{an^2}{an^2+bn+c}$. Finally, Speedup(p) $\approx \frac{1}{\frac{bn+c}{an^2+bn+c} + \frac{1-\frac{bn+c}{an^2+bn+c}}{p}}$, and for n important, Speedup(p) $\approx \frac{1}{\frac{b}{an} + \frac{1}{p}}$. For large p, $\lim_{p \to \infty} Speedup(p) = \frac{an}{b}$ and is therefore larger the larger n is. The

overall idea is that the algorithm is structurally parallelizable, adding threads seems worthwhile. For p = 4, and a large home array, Speedup(4) should theoretically be close to $\frac{1}{\frac{b}{an}+\frac{1}{4}} \approx 4$.

Thus, if we have four cores that allow threads to execute truly simultaneously, performance should be better than the brute-force version without parallelization. However, even with this parallelism, execution time should not be better than the version with an O(n) slicing window.

Space complexity

The candy vector is passed by constant reference, meaning it is not copied and its memory, allocated on the heap, simply represents the input in O(n). The global variables (bestSum, bestStart, bestEnd) occupy a constant space, or O(1). In the parallel region, each thread has a fixed number of local variables (localBestSum, localStart, localEnd, sum), which represents O(p) total memory for p threads; however, p is considered constant, so this remains O(1). There is no dynamic allocation, no auxiliary array, and no recursion. Thus, the total space complexity used is O(n).

Performances

Time performances

Spécifications du processeur

Nb. de cœurs	2
Nombre total de threads	4
Fréquence Turbo maxi	4.00 GHz
Fréquence de la technologie Intel® Turbo Boost 2.0†	4.00 GHz
Fréquence de base	3.50 GHz
Cache	4 MB
Vitesse du bus	4 GT/s
PDT	28 W
PDT-down configurable	23 W

Fig.12 : Excerpt from the product sheet for the core of the device I am working on

My machine's processor is an Intel Core i7-7567U. It has two physical cores and supports four logical threads thanks to Hyper-Threading technology. This means that only two threads can actually run in parallel on separate hardware units, while the other two logical threads share the internal resources of the same cores. When parallelizing with OpenMP, using two or four threads allows for full utilization of the physical cores and generally represents the best compromise in terms of efficiency. By default, openMP uses the number of available cores, in our case 4 threads. Beyond that, gains can be limited by resource sharing and the cost of context switching. Thus, the speedup observed in practice will be lower than the ideal theoretical speedup.

We benchmark to see the mean, median, standard deviation, and coefficient of variation of the execution times. We are interested in finding the performances of the parallelized brute-force sequence (best_sequence_parallel) for different numbers of threads depending on different input sizes. We also see the performance of the complete algorithm depending on the number of threads for different input sizes.

Benchmark	Time	CPU	Iterations
BM_BestSequenceParallel/1/100/repeats:20_mean	3265 ns	3158 ns	20
BM_BestSequenceParallel/1/100/repeats:20_median	3083 ns	3083 ns	20
BM_BestSequenceParallel/1/100/repeats:20_stddev	787 ns	329 ns	20
BM_BestSequenceParallel/1/100/repeats:20_cv	24.09 %	10.43 %	20
BM_BestSequenceParallel/2/100/repeats:20_mean	3961 ns	3744 ns	20
BM_BestSequenceParallel/2/100/repeats:20_median	3445 ns	3445 ns	20
BM_BestSequenceParallel/2/100/repeats:20_stddev	1439 ns	885 ns	20
BM_BestSequenceParallel/2/100/repeats:20_cv	36.33 %	23.65 %	20
BM_BestSequenceParallel/4/100/repeats:20_mean	7685 ns	5991 ns	20
BM_BestSequenceParallel/4/100/repeats:20_median	4034 ns	4021 ns	20
BM_BestSequenceParallel/4/100/repeats:20_stddev	10478 ns	5797 ns	20
BM_BestSequenceParallel/4/100/repeats:20_cv	136.35 %	96.77 %	20
BM_BestSequenceParallel/8/100/repeats:20_mean	67073 ns	28390 ns	20
BM_BestSequenceParallel/8/100/repeats:20_median	67287 ns	28314 ns	20
BM_BestSequenceParallel/8/100/repeats:20_stddev	3128 ns	827 ns	20
BM_BestSequenceParallel/8/100/repeats:20_cv	4.66 %	2.91 %	20
BM_BestSequenceParallel/16/100/repeats:20_mean	224325 ns	35485 ns	20
BM_BestSequenceParallel/16/100/repeats:20_median	190638 ns	34593 ns	20
BM_BestSequenceParallel/16/100/repeats:20_stddev	58051 ns	1984 ns	20
BM_BestSequenceParallel/16/100/repeats:20_cv	25.88 %	5.59 %	20

Fig.13 : Benchmark of best_parallel_sequence for different numbers of threads for an input of size 100

BM_BestSequenceParallel/1/1000/repeats:20_mean	32600 ns	32419 ns
BM_BestSequenceParallel/1/1000/repeats:20_median	31926 ns	31919 ns
BM_BestSequenceParallel/1/1000/repeats:20_stddev	3925 ns	3321 ns
BM_BestSequenceParallel/1/1000/repeats:20_cv	12.04 %	10.24 %
BM_BestSequenceParallel/2/1000/repeats:20_mean	19233 ns	18885 ns
BM_BestSequenceParallel/2/1000/repeats:20_median	18016 ns	18015 ns
BM_BestSequenceParallel/2/1000/repeats:20_stddev	4120 ns	3293 ns
BM_BestSequenceParallel/2/1000/repeats:20_cv	21.42 %	17.44 %
BM_BestSequenceParallel/4/1000/repeats:20_mean	334407 ns	202492 ns
BM_BestSequenceParallel/4/1000/repeats:20_median	33439 ns	33437 ns
BM_BestSequenceParallel/4/1000/repeats:20_stddev	1377498 ns	792267 ns
BM_BestSequenceParallel/4/1000/repeats:20_cv	411.92 %	391.26 %
BM_BestSequenceParallel/8/1000/repeats:20_mean	101017 ns	34596 ns
BM_BestSequenceParallel/8/1000/repeats:20_median	88655 ns	33692 ns
BM_BestSequenceParallel/8/1000/repeats:20_stddev	38971 ns	2637 ns
BM_BestSequenceParallel/8/1000/repeats:20_cv	38.58 %	7.62 %
BM_BestSequenceParallel/16/1000/repeats:20_mean	217856 ns	36962 ns
BM_BestSequenceParallel/16/1000/repeats:20_median	199460 ns	35868 ns
BM_BestSequenceParallel/16/1000/repeats:20_stddev	59036 ns	3405 ns
BM_BestSequenceParallel/16/1000/repeats:20_cv	27.10 %	9.21 %

Fig.14: Benchmark of best_parallel_sequence for different numbers of threads for an input of size 1000

BM_BestSequenceParallel/1/10000/repeats:20_mean	314031 ns	313972 ns
BM_BestSequenceParallel/1/10000/repeats:20_median	301988 ns	301959 ns
BM_BestSequenceParallel/1/10000/repeats:20_stddev	35975 ns	35912 ns
BM_BestSequenceParallel/1/10000/repeats:20_cv	11.46 %	11.44 %
BM_BestSequenceParallel/2/10000/repeats:20_mean	163065 ns	162987 ns
BM_BestSequenceParallel/2/10000/repeats:20_median	161399 ns	161339 ns
BM_BestSequenceParallel/2/10000/repeats:20_stddev	12636 ns	12589 ns
BM_BestSequenceParallel/2/10000/repeats:20_cv	7.75 %	7.72 %
BM_BestSequenceParallel/4/10000/repeats:20_mean	130800 ns	128157 ns
BM_BestSequenceParallel/4/10000/repeats:20_median	126005 ns	125242 ns
BM_BestSequenceParallel/4/10000/repeats:20_stddev	20028 ns	18133 ns
BM_BestSequenceParallel/4/10000/repeats:20_cv	15.31 %	14.15 %
BM_BestSequenceParallel/8/10000/repeats:20_mean	223283 ns	87761 ns
BM_BestSequenceParallel/8/10000/repeats:20_median	223952 ns	88540 ns
BM_BestSequenceParallel/8/10000/repeats:20_stddev	13008 ns	3914 ns
BM_BestSequenceParallel/8/10000/repeats:20_cv	5.83 %	4.46 %
BM_BestSequenceParallel/16/10000/repeats:20_mean	341122 ns	62200 ns
BM_BestSequenceParallel/16/10000/repeats:20_median	327313 ns	59400 ns
BM_BestSequenceParallel/16/10000/repeats:20_stddev	28965 ns	5920 ns
BM_BestSequenceParallel/16/10000/repeats:20_cv	8.49 %	9.52 %

Fig.15 : Benchmark of best_parallel_sequence for different numbers of threads for an input of size 10000

BM_FullPipelineParallel/4/100/repeats:20_mean	54483 ns	44612 ns
BM_FullPipelineParallel/4/100/repeats:20_median	40488 ns	38503 ns
BM_FullPipelineParallel/4/100/repeats:20_stddev	43306 ns	23549 ns
BM_FullPipelineParallel/4/100/repeats:20_cv	79.49 %	52.79 %
BM_FullPipelineParallel/4/1000/repeats:20_mean	363085 ns	263779 ns
BM_FullPipelineParallel/4/1000/repeats:20_median	209208 ns	204470 ns
BM_FullPipelineParallel/4/1000/repeats:20_stddev	491097 ns	188316 ns
BM_FullPipelineParallel/4/1000/repeats:20_cv	135.26 %	71.39 %
BM_FullPipelineParallel/4/10000/repeats:20_mean	3665762 ns	2428403 ns
BM_FullPipelineParallel/4/10000/repeats:20_median	1852349 ns	1405447 ns
BM_FullPipelineParallel/4/10000/repeats:20_stddev	3165800 ns	1481946 ns
BM_FullPipelineParallel/4/10000/repeats:20_cv	86.36 %	61.03 %

Fig.16 : Benchmark of the full pipeline for 4 threads and different size of input

For small inputs, parallelism costs more than it benefits. For inputs of size 1000, the best time is when we use 2 threads (with a median of 18016 ns). For inputs of size 1000, the best time is when we use 4 threads (with a median of 126005 ns), but 2 threads are also suitable. Coefficients of variation are often quite significant, however, and there are sometimes consequent outliners (for `BM_BestSequenceParallel/4/1000`, we get 412% for exemple).

Overall, the times for the parallelized brute-force sequence are significantly worse than for the sequential sliding window, and the same is true for the complete pipeline.

Performances

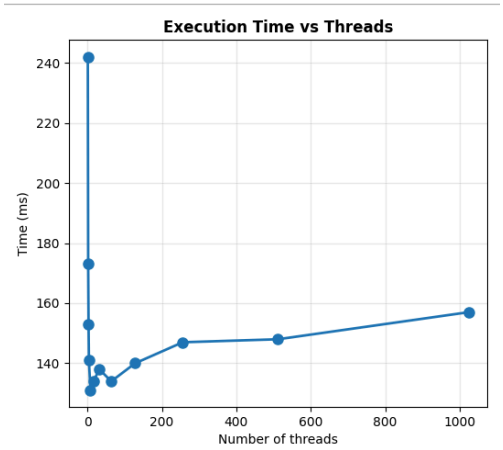

Fig.17 : Execution time of the complete pipeline versus the number of threads for an entry of size 10000

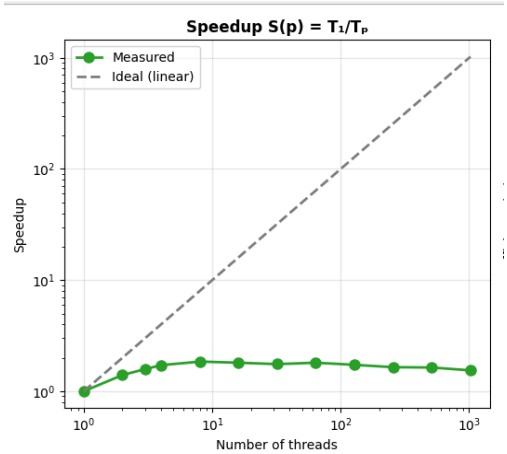

Fig. 18 : Speedup of the complete pipeline versus the number of threads for an entry of size 10000

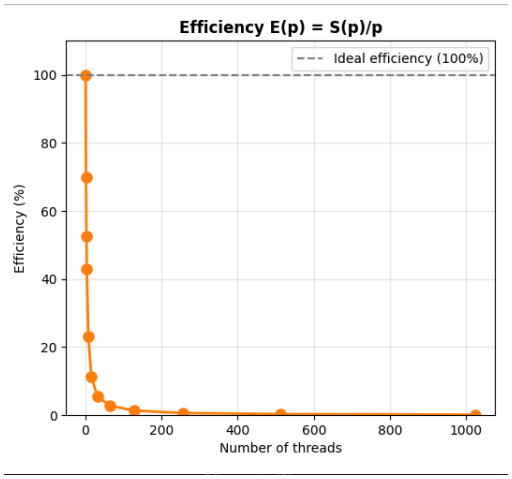

Fig. 19 : Efficiency versus the number of threads for an entry of size 10000

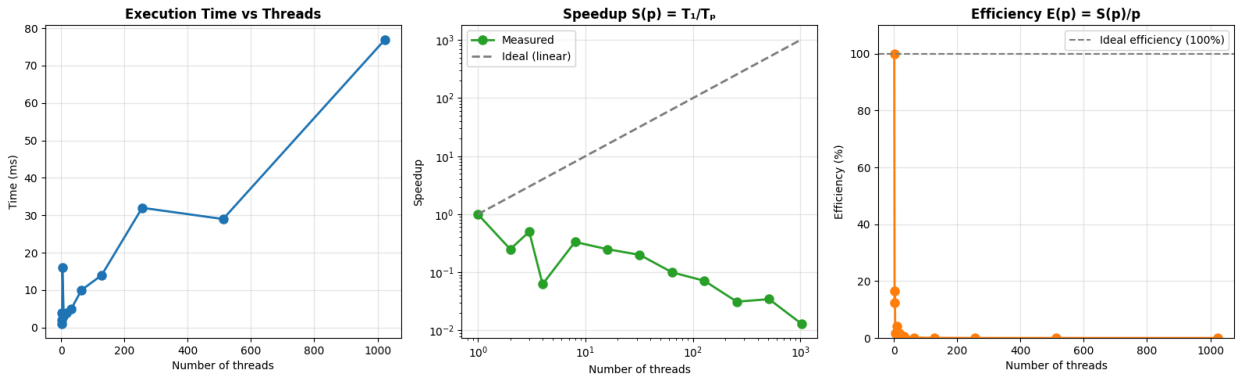

Fig. 19 : Execution time, speedup and efficiency of the complete pipeline versus the number of threads for an entry of size 500

As mentioned previously, early stopping allows the 1-thread algorithm to achieve excellent performance, which is why we will test the algorithm on examples that require traversing all houses. Otherwise, the time for one thread T_1 is better than all the times T□ for p threads.

We created —using a coding agent, due to time constraints, sorry, I could have displayed these graphs using the information returned by the benchmark, for example— a script that generates three complementary graphs to analyze parallelization. The first graph (Execution Time) shows how the time decreases with more threads : ideally, we expect a decreasing curve following a hyperbola $T(p) = T_1/p$, where p is the number of threads. The second graph (Speedup log-log) shows the measured speedup $S(p) = T_1/T$□ as a function of the number of threads : ideally, the speedup is linear, meaning that x threads are x times faster than 1 thread. The third graph (Efficiency) calculates the efficiency $E(p) = S(p)/p \times 100\%$, that is, how much each thread is actually working : ideally, the efficiency remains constant, close to 100%.

In the case where our input has 10,000 elements, we see that the execution time decreases and is minimal for 16 threads before increasing again. The speedup increases slightly before plateauing and even decreasing. It is far from the ideal speedup, which does not take overhead into account. Efficiency, meanwhile, drops rapidly, from 100% to nearly 22% for 16 threads and down to almost 0% for 1024 threads. Indeed, the critical section (#pragma omp critical) forces threads to wait their turn, which negates the benefits of parallelization. We can therefore look for the inflection point where efficiency falls below 50%, this is generally the optimal number of threads. Beyond that, adding threads costs more than the minimal gains it provides. For this program, the default choice of 4 threads in OpenMP is correct, 8 or even 2 threads could also be considered.

If our input contains only 500 elements, we see that the execution time decreases slightly and is minimal when executed sequentially. It remains acceptable up to 16 threads, but then increases significantly. The speedup clearly demonstrates that there is a performance decrease with the number of threads. Efficiency, on the other hand, drops extremely rapidly, going from 100% to less than 5% for 4 threads. Indeed, the cost of creating threads unsurprisingly completely negates the benefits of parallelization for an input of this size.

```
computer@computer:~/Bureau/Elise_asuppr/Nvidia/trick_or_treat/openMP$ perf stat ./test 1
Start at home 1 and go to home 10000 getting 1 pieces of candy

=== RESULTS ===
File: ../input_pirecas.txt
Homes: 10000
Max candy: 1000
Result: Start=1, End=10000, Sum=1
Total time: 281 ms

 Performance counter stats for './test 1':

            284,33 msec task-clock                       #    0,995 CPUs utilized
                11      context-switches                 #   38,687 /sec
                 1      cpu-migrations                   #    3,517 /sec
               149      page-faults                      #  524,036 /sec
       887 144 426      cycles                           #    3,120 GHz
     1 763 034 403      instructions                     #    1,99  insn per cycle
       352 357 612      branches                         #    1,239 G/sec
            45 477      branch-misses                    #    0,01% of all branches

       0,285654560 seconds time elapsed

       0,283198000 seconds user
       0,002001000 seconds sys
```

Fig.20 : Performance statistics while running the whole pipeline with 1 thread

```
computer@computer:~/Bureau/Elise_asuppr/Nvidia/trick_or_treat/openMP$ perf stat -e L1-dcache-loads,L1-dcache-load-misses,LLC-loads,LLC-load-misses ./test 1
Start at home 1 and go to home 10000 getting 1 pieces of candy

=== RESULTS ===
File: ../input_pirecas.txt
Homes: 10000
Max candy: 1000
Result: Start=1, End=10000, Sum=1
Total time: 283 ms

 Performance counter stats for './test 1':

       953 564 617      L1-dcache-loads
         1 561 937      L1-dcache-load-misses            #    0,16% of all L1-dcache accesses
            52 614      LLC-loads
            14 929      LLC-load-misses                  #   28,37% of all LL-cache accesses

       0,286511577 seconds time elapsed

       0,285499000 seconds user
       0,001005000 seconds sys
```

Fig.21 : Cache performance while running the whole pipeline with 1 thread

```
Samples: 1K of event 'cycles:P', Event count (approx.): 797404717
Overhead  Command  Shared Object        Symbol
  78,44%  test     test                 [.] best_sequence_parallel(std::vector<int, std::allocator<int> > const&, int, int, int&, int&
  19,52%  test     test                 [.] std::vector<int, std::allocator<int> >::operator[](unsigned long) const
   0,36%  test     [kernel.kallsyms]    [k] __irqentry_text_end
   0,09%  test     libstdc++.so.6.0.30  [.] std::istream::sentry::sentry(std::istream&, bool)@plt
   0,09%  test     libstdc++.so.6.0.30  [.] std::istreambuf_iterator<char, std::char_traits<char> > std::num_get<char, std::istreambu
   0,09%  test     [kernel.kallsyms]    [k] __update_load_avg_se
   0,09%  test     [kernel.kallsyms]    [k] lapic_next_deadline
   0,09%  test     [kernel.kallsyms]    [k] hrtimer_interrupt
   0,09%  test     [kernel.kallsyms]    [k] irq_exit_rcu
   0,09%  test     [kernel.kallsyms]    [k] native_irq_return_iret
   0,09%  test     [kernel.kallsyms]    [k] __update_blocked_fair
   0,09%  test     [kernel.kallsyms]    [k] xas_load
   0,09%  test     [kernel.kallsyms]    [k] hrtimer_active
   0,09%  test     [kernel.kallsyms]    [k] zap_pte_range
   0,09%  test     [kernel.kallsyms]    [k] copy_mc_enhanced_fast_string
   0,08%  test     ld-linux-x86-64.so.2 [.] check_match
   0,08%  test     ld-linux-x86-64.so.2 [.] do_lookup_x
   0,08%  test     ld-linux-x86-64.so.2 [.] strcmp
   0,07%  test     [kernel.kallsyms]    [k] memset_orig
   0,07%  test     [kernel.kallsyms]    [k] __do_sys_newfstatat
   0,07%  test     [kernel.kallsyms]    [k] __d_lookup_rcu
   0,06%  test     [kernel.kallsyms]    [k] nd_jump_root
   0,04%  test     [kernel.kallsyms]    [k] security_file_alloc
   0,03%  test     ld-linux-x86-64.so.2 [.] _dl_sysdep_start
   0,02%  test     [kernel.kallsyms]    [k] ___slab_alloc
   0,02%  test     [kernel.kallsyms]    [k] perf_output_begin
```

Fig.22 : Distribution of the use of the CPU while running the whole pipeline with 1 thread

```
computer@computer:~/Bureau/Elise_asuppr/Nvidia/trick_or_treat/openMP$ perf stat -e L1-dcache-loads,L1-dcache-load-misses,LLC-loads,LLC-load-misses
Start at home 1 and go to home 10000 getting 1 pieces of candy

=== RESULTS ===
File: ../input_pirecas.txt
Homes: 10000
Max candy: 1000
Result: Start=1, End=10000, Sum=1
Total time: 335 ms

 Performance counter stats for './test':

       955 260 760      L1-dcache-loads
         1 976 710      L1-dcache-load-misses           #    0,21% of all L1-dcache accesses
           154 744      LLC-loads
            10 301      LLC-load-misses                 #    6,66% of all LL-cache accesses

       0,349134843 seconds time elapsed

       0,485072000 seconds user
       0,003911000 seconds sys
```

Fig.23 : Performance statistics while running the whole pipeline with 4 thread

```
computer@computer:~/Bureau/Elise_asuppr/Nvidia/trick_or_treat/openMP$ perf stat ./test
Start at home 1 and go to home 10000 getting 1 pieces of candy

=== RESULTS ===
File: ../input_pirecas.txt
Homes: 10000
Max candy: 1000
Result: Start=1, End=10000, Sum=1
Total time: 200 ms

 Performance counter stats for './test':

            483,78 msec task-clock                       #    2,367 CPUs utilized
               545      context-switches                 #    1,127 K/sec
                 5      cpu-migrations                   #   10,335 /sec
               156      page-faults                      #  322,463 /sec
     1 455 172 952      cycles                           #    3,008 GHz
     1 773 944 150      instructions                     #    1,22  insn per cycle
       355 113 872      branches                         #  734,045 M/sec
            69 556      branch-misses                    #    0,02% of all branches

       0,204345756 seconds time elapsed

       0,479206000 seconds user
       0,004811000 seconds sys
```

Fig.24 : Cache performance while running the whole pipeline with 4 thread

```
Samples: 1K of event 'cycles:P', Event count (approx.): 1315048236
Overhead  Command  Shared Object          Symbol
  64,31%  test     test                   [.] best_sequence_parallel(std::vector<int, std::allocator<int> > const&, int, int, int&, int
  22,91%  test     test                   [.] std::vector<int, std::allocator<int> >::operator[](unsigned long) const
  10,39%  test     libgomp.so.1.0.0       [.] 0x00000000000207b8
   0,55%  test     libgomp.so.1.0.0       [.] 0x00000000000207ba
   0,43%  test     libgomp.so.1.0.0       [.] 0x00000000000207be
   0,27%  test     libgomp.so.1.0.0       [.] 0x00000000000207c3
   0,17%  test     libgomp.so.1.0.0       [.] 0x00000000000207c8
   0,11%  test     [kernel.kallsyms]      [k] __irqentry_text_end
   0,06%  test     libstdc++.so.6.0.30    [.] std::istreambuf_iterator<char, std::char_traits<char> > std::num_get<char, std::istreambu
   0,06%  test     libstdc++.so.6.0.30    [.] std::istream::sentry::sentry(std::istream&, bool)@plt
   0,05%  test     libc.so.6              [.] __unregister_atfork
   0,05%  test     test                   [.] __gnu_cxx::__enable_if<std::__is_scalar<int>::__value, void>::__type std::__fill_a1<int*,
   0,05%  test     [kernel.kallsyms]      [k] uprobe_deny_signal
   0,05%  test     [kernel.kallsyms]      [k] native_read_msr
   0,05%  test     [kernel.kallsyms]      [k] clockevents_program_event
   0,05%  test     [kernel.kallsyms]      [k] unlink_anon_vmas
   0,05%  test     ld-linux-x86-64.so.2   [.] do_lookup_x
   0,05%  test     ld-linux-x86-64.so.2   [.] _dl_lookup_symbol_x
   0,05%  test     [kernel.kallsyms]      [k] next_uptodate_folio
   0,05%  test     [kernel.kallsyms]      [k] vma_interval_tree_remove
   0,04%  test     [kernel.kallsyms]      [k] do_anonymous_page
   0,04%  test     [kernel.kallsyms]      [k] set_root
   0,04%  test     [kernel.kallsyms]      [k] entry_SYSRETQ_unsafe_stack
   0,03%  test     [kernel.kallsyms]      [k] generic_permission
   0,03%  test     [kernel.kallsyms]      [k] x64_sys_call
   0,02%  test     [kernel.kallsyms]      [k] set_pte_range
   0,01%  test     [kernel.kallsyms]      [k] __perf_addr_filters_adjust
   0,01%  test     [kernel.kallsyms]      [k] memset_orig
```

Fig.25 : Distribution of the use of the CPU while running the whole pipeline with 4 thread

We tested the brute-force algorithm on an input called "input_10000_pirecas.txt", which requires traversing 10,000 houses, representing one of the worst-case scenarios for the algorithm.

We observed (Fig. 20 & 23) that the 4-thread version showed a 1.4 speedup compared to the sequential execution (1 thread). For 1 thread, 0.99 CPUs are used, which is optimal for single-threaded code, while for 4 threads, only 2.37 CPUs are used despite 4 CPUs being available. The parallel efficiency is therefore only 59% (2.37/4). The number of instructions per cycle confirms this inefficiency. For 1 thread, 1.99 instructions are processed per cycle compared to 1.22 for 4 threads. This decrease indicates that not all threads are working efficiently; they are spending time waiting, probably in the critical section (#pragma omp critical) required for synchronization. Cache performance is good and branch misses remain low; the bottleneck is not related to memory or branch predictions. The number of context switches reveals the synchronization overhead. There are 1100 switches/s for 4 threads compared to 38 switches/s for 1 thread. Threads are frequently interrupted by the operating system, presumably during waits on the critical section. Analysis with perf report (Fig. 22 & 25) confirms this overhead : for 1 thread, 78% of CPU time is dedicated to the algorithm and 20% to memory accesses, with the remainder for the system ; for 4 threads, 27% (10.39 + 8.55 + 8.43) of CPU time is consumed by the OpenMP library (libgomp) for synchronization management. Over an execution time of 200ms, this represents approximately 54ms of pure overhead.

When we test the sequential program versus the 4-threaded version with the same random input, we observe that instead that the speedup can vary, sometimes we even experience a slowdown compared to the non-parallelized version. This variability stems from the trade-off between the overhead of synchronization (thread creation, management, and synchronization) and the gain provided by parallelism, which is highly dependent on the input size, of course, as well as the possibility of an early stop (if a sequence is found that yields maxCandy; if there is only one thread, the algorithm terminates there).

2) Sliding window algorithm with file reading by multiple threads

Principle

We've seen that the sliding window allows for linear complexity, which is optimal. The part of the code that also takes time is actually the file reading, which is also linear (but where early stopping isn't possible). One idea that comes to mind is to try parallelizing the file reading. The `read_input` function reads a file containing the number of candies per house and stores these values in a `candy` vector. The idea behind parallelization is to use multiple threads to speed up processing when the vector is very large. However, it's not possible to use the same `ifstream` object from multiple threads simultaneously, as this would lead to undefined behavior, so it's impossible to read the file directly in parallel. The solution is to separate the process into two steps: first, read the file sequentially into a buffer, then perform the copying—and processing, although there isn't any here—of the data in parallel. This solution would be much more useful if it didn't just involve copying the elements but also processing them simultaneously. We will implement this solution for testing purposes.

To enable this parallelization, it seems useful to use an intermediate buffer called `temp`. This buffer temporarily stores the data read from the file so that multiple threads can then copy it simultaneously. Since the maximum size is small (at most 10,000 houses, or about 40 KB), this buffer could be placed on the stack. Alternatively, it can be allocated on the heap.

Implementation with OpenMP

```
      bool read_input_openmp(const std::string &filename, int &homes, int
&maxCandy, std::vector<int> &candy) {

         std::ifstream file(filename);

         if (!file) {
             std::cerr << "Error: could not open " << filename <<
std::endl;
             return false;
         }

         file >> homes;
         file >> maxCandy;

         if (homes <= 0) {
             file.close();
             return true;
         }
```

```
        // Sequential reading of the file into a temporary buffer on the
stack
        int temp[10000]; // there are at most 10000 houses, so this is
safe
       //Other option : Sequential reading of the file into a temporary
buffer on the heap
       //std::vector<int> temp(homes);

        for (int i = 0; i < homes; i++) {
            file >> temp[i];
        }

        file.close();

        candy.resize(homes);

        // Parallel filling
        #pragma omp parallel for
        for (int i = 0; i < homes; i++) {
            candy[i] = temp[i];
        }

        return true;
    }
```

Fig.26 : The read_input_openmp code

The data flow occurs in three steps: the file on disk is read into the temp buffer (on the stack or heap), then the threads copy the data to fill the candy vector (heap). The file reading remains sequential, but the copying can be distributed across multiple threads. Each thread writes to distinct indices of the vector, which avoids memory conflicts, and contiguous data access allows for efficient use of the CPU cache.

Performances

Comparison

	Stack	**Heap**	**Without parallelization**
Time elapsed	0.014630831	0.023681025	0.00302247
CPUs	2.443	1.807	0.850
Context-switches	28 (654/sec)	11 (289/sec)	0

Instruction per cycle	0.18	0.19	1.54
L1-dcache misses	1.76%	1.62%	1.59%
LLC-load-misses	32.72% of 30214	32.58% of 27944	29.71% of 22414

One would expect the stack to be slightly faster. This is not the case, but the performance with the stack and the heap is similar, which is not surprising.

Conclusion

For a number of elements on the order of 10.000, parallelization reduces performance. Parallelization could become advantageous if processing needed to be applied to each element of a very large vector. However, simply copying 10,000 integers is very fast on the CPU, and the real bottleneck is reading the file from disk.

3) Sliding window algorithm with parallel reading and calculation of the sequence

Principle

Another approach, using a pipeline parallelism, would be to simultaneously read the file and calculate the best house sequence. This can be achieved using multiple threads in a producer-consumer model, but with only one producer and one consumer. One thread reads the file and stores the values in a buffer, while the other thread consumes this buffer to update the best sequence in real time.

The read thread could read each value from the file and push it into a queue. The calculation thread could retrieve these values as they are available and update the sum, start, and end of the optimal sequence, as well as bestSum. This allows the calculation to begin as soon as the first values are available, without waiting for the entire file to be read.

The producer must not write if the buffer is full, and the consumer must not read if the buffer is empty. Therefore, a mutex or a variable condition must be used to ensure that reading and writing are safe. The read thread notifies the calculation thread of each new value and signals the end of the file when all the data has been read. The computation thread then applies the sliding window algorithm in real time: it adds the new value, removes the initial values if the sum exceeds maxCandy, and updates the best sequence found.

Performances

This approach allows for simultaneous reading and calculation, which makes better use of the CPU and can compensate for the wait time while reading the file. For very large files or computationally intensive calculations, this parallelization could undoubtedly reduce the overall execution time. However, for a small file like ours, the gain is almost negligible.

Conclusion

We can try to parallelize the algorithm with the sliding window. For example, we can have threads that look at different home ranges and apply the sliding window to them, managing overlap between ranges to handle windows crossing boundaries.

We can also try to improve brute force. We can keep the initial iteration of each index but then perform a binary search.

However, the sliding window algorithm already has O(n) time complexity. A linear traversal cannot be improved when each element must be examined at least once. Adding more algorithmic sophistication would likely increase code complexity, memory consumption, or synchronization cost, without any real practical benefit.

The best approach, therefore, is to retain the sliding window algorithm to guarantee linear complexity, and then use compilation with g++ -03 to allow the compiler to apply advanced optimizations. This results in a code that is both theoretically optimal and efficiently optimized at the machine level.